## Name: BYUMVUHORE Aimable

## 1. ERD Schema (Entity-Relationship Diagram)

For the ERD, I need to define the tables (entities) and their relationships in the system.

**Example Entities:**

1. **User**
    - user_id (PK)
    - first_name
    - last_name
    - email (unique)
    - password_hash
    - role (admin/user/etc.)
    - created_at
    - updated_at
2. **Product**
    - product_id (PK)
    - name
    - description
    - price
    - category_id (FK to Category)
    - created_at
    - updated_at

3. **Category**
    - category_id (PK)
    - name
    - description
4. **Order**
    - order_id (PK)
    - user_id (FK to User)
    - order_date
    - total_amount
    - status (pending, completed, canceled)
    - created_at
    - updated_at
5. **OrderItem**
    - order_item_id (PK)
    - order_id (FK to Order)
    - product_id (FK to Product)
    - quantity
    - price
6. **Payment**
    - payment_id (PK)
    - order_id (FK to Order)
    - payment_method (credit_card, paypal)

    - payment_status (successful, failed)
    - amount
    - payment_date

7. **ShippingAddress**
    - shipping_address_id (PK)
    - user_id (FK to User)
    - address_line_1
    - address_line_2
    - city
    - state
    - zip_code
    - country
8. **Inventory**
    - inventory_id (PK)
    - product_id (FK to Product)
    - stock_quantity
    - restock_date

**Relationships:**

- **User** can have many **Orders**.
- **Order** can have many **OrderItems**.
- **Product** belongs to one **Category**.
- **Product** can be part of many **OrderItems**.

- **Order** has one **Payment**.
- **User** can have many **ShippingAddresses**.
- **Product** has one **Inventory**.

For ERD, I use rectangles for entities, lines for relationships, and crow's feet to represent cardinality (one-to-many, many-to-many).

## 2. System Architecture

For system architecture, I'll assume a basic 3-layer architecture (presentation, business logic, data storage), which is common for web applications.

**Basic Components:**

- **Frontend**: React or Vue.js (SPA, interacts with backend via REST API)
- **Backend**: Java Spring Boot or Node.js (express.js or NestJS)
- **Database**: PostgreSQL (DB server)
- **Authentication**: JWT or OAuth2
- **External APIs**: (payment gateway, email services, etc.)
- **Cloud Provider**: AWS or GCP (if required for scalability)

**Architecture Flow:**

1. **User** interacts with the **Frontend** (React/Vue).
2. The **Frontend** sends requests (REST API calls) to the **Backend**.
3. The **Backend** processes the logic and interacts with **PostgreSQL** to store/retrieve data.
4. After processing, the **Backend** sends a response back to the **Swagger**.

This represents components as boxes and draws arrows for data flow between them (use directional arrows).

## 3. POJOs (Plain Old Java Objects)

The POJOs represent the data models for the backend. Here's an example of what the POJOs might look like based on the ERD:

**Example POJOs (in Java):**

1. **User.java**:

```
public class User {
    private Long userId;
    private String firstName;
    private String lastName;
    private String email;
    private String passwordHash;
    private String role;
    private Date createdAt;
    private Date updatedAt;
}
```

2. **Product.java**:

```
public class Product {
    private Long productId;
    private String name;
    private String description;
    private BigDecimal price;
    private Long categoryId;
    private Date createdAt;
    private Date updatedAt;
}
```

3. **Order.java**:

```
public class Order {
    private Long orderId;
    private Long userId;
    private Date orderDate;
    private BigDecimal totalAmount;
    private String status;
    private Date createdAt;
```

```
    private Date updatedAt;
}
```

Each entity will have corresponding POJOs that map to the database tables.

## 4. Choice of Tech Stack

1. **Frontend**:
    - **Framework**: Swagger Documentation
2. **Backend**:
    - **Framework**: Spring Boot (Java)
    - **Database**: PostgreSQL
    - **ORM**: Hibernate (for Spring Boot),
    - **Authentication**: JWT or OAuth2 for user login
    - **API**: RESTful API (JSON format)
3. **Database**:
    - **PostgreSQL** for data storage.
4. **Security**:
    - **JWT Authentication** (for stateless sessions).
    - **Encryption**: Use bcrypt for password hashing.
    - **Authorization**: Role-based access control (RBAC).
    - **Input Validation**: Prevent SQL injection and XSS attacks.
    - **CORS**: Set up Cross-Origin Resource Sharing (CORS) properly to secure your API endpoints.

## 5. Security Considerations

- **Password Storage**: Store passwords securely by hashing them using algorithms like **bcrypt**.
- **Rate Limiting**: Use API rate limiting to prevent abuse.
- **JWT Expiry**: Set an expiration time for JWT tokens to improve security.
- **Input Sanitization**: Validate all inputs to prevent SQL injection and cross-site scripting (XSS).

## 6. Outline of Endpoints to Develop

Here's a list of possible **REST API endpoints** based on the requirements:

**Authentication Endpoints:**

- `POST /api/auth/register` (User registration)
- `POST /api/auth/login` (User login)
- `POST /api/auth/logout` (User logout)

**User Endpoints:**

- `GET /api/users/{userId}` (Get user details)
- `PUT /api/users/{userId}` (Update user details)
- `DELETE /api/users/{userId}` (Delete user)

**Product Endpoints:**

- `GET /api/products` (List all products)
- `POST /api/products` (Create a new product)
- `GET /api/products/{productId}` (Get product details)
- `PUT /api/products/{productId}` (Update product)

- `DELETE /api/products/{productId}` (Delete product)

**Order Endpoints:**

- `GET /api/orders` (List user orders)
- `POST /api/orders` (Create a new order)
- `GET /api/orders/{orderId}` (Get order details)

**Payment Endpoints:**

- `POST /api/payments` (Initiate payment for an order)
- `GET /api/payments/{paymentId}` (Get payment details)

**Shipping Address Endpoints:**

- `GET /api/addresses/{userId}` (List shipping addresses)
- `POST /api/addresses` (Add a shipping address)
- `PUT /api/addresses/{addressId}` (Update an address)
- `DELETE /api/addresses/{addressId}` (Delete an address)